NETCALC

Calculatrice Réseau IPv4

Document Pédagogique

- Concepts fondamentaux du sous-réseautage IPv4
- Calcul du masque, CIDR, adresses réseau et diffusion
- Concepts fondamentaux du sous-réseautage
- Technique VLSM avec des exemples
- Implémentation compléte en Python commenté
- Quelques exercices corrigés

Projet : Calculatrice réseau en Python

Réalisé par Serigne Cheikh Mbacké Kane

Table des matières

1. Introduction & Objectifs Pédagogiques 3
2. Rappels : Le Système Binaire 4
2.1 Conversion décimal ↔ binaire 4
2.2 Les opérations bit à bit — ET et OU logique 4
3. Concepts Fondamentaux du Réseau IPv4 5
3.1 L'adresse IP 5
3.2 Le masque de sous-réseau 5
3.3 La notation CIDR 5
Comment trouver le nombre de bits de la partie hôte ? 5
Tableau récapitulatif du masque et du CIDR 6
3.4 L'adresse réseau — Opération ET logique (AND) 6
3.5 L'adresse de diffusion (Broadcast) — Opération OU logique 7
3.6 Plage d'adresses utilisables 7
4. Le VLSM — Découpage en Sous-réseaux 8
4.1 Pourquoi le VLSM ? 8
4.2 Méthode de découpage VLSM — Étape par étape 8
4.3 Exemple de découpage — Réseau 192.168.10.0 9
5. Implémentation Python — Netcalc 10
5.1 Architecture du programme 10
5.2 Fonctions de saisie et de validation 11
5.3 Calcul du masque — mask_en_decimal() et conv_mask_decimal_vers_binaire() 12
5.4 Fonction centrale — calcul_info_sous_reseau() 13
5.5 Un sous-réseau — reseau_simple() 14
5.6 Plusieurs sous-réseaux (VLSM) 15
5.7 Fonctions d'affichage — formater_liste() et affichage_infos_sous_reseau() 16
5.8 Menu principal — main() 16
6. Exemples Complets Résolus 17
Exemple 1 — Réseau simple (mode 1) 17
Exemple 2 — VLSM complet (mode 2) 18
7. Exercices Corrigés 19
Exercice 1 — Calcul simple 19
Exercice 2 — Interprétation du masque binaire 19
Exercice 3 — VLSM 20
8. Conclusion 19

1. Introduction & Objectifs Pédagogiques

Ce document est un guide pédagogique complet sur le **calcul de sous-réseaux IPv4**. Il accompagne le programme Python **Netcalc**, une calculatrice réseau qui automatise ces calculs. L'objectif est double : comprendre les concepts théoriques et voir comment ils se traduisent en code.

❖ Dans cette perspective de mieux comprendre les concepts théoriques du sous-réseautage, nous avons choisi de n'utiliser aucune bibliothèque spécialisée (une fois les bases acquises et maitrisées, ces bibliothèques vous seront d'une grande aide pour mettre en place un programme plus robuste et plus optimisé)
Exemple de bibliothèques spécialisées : ipaddress, netaddr…

#	Objectif pédagogique	Section
1	Comprendre la structure d'une adresse IPv4 et du masque	3.1
2	Maîtriser la notation CIDR et le calcul du masque	3.2 et 3.3
3	Savoir calculer manuellement l'adresse réseau et de diffusion	3.4 et 3.5
4	Comprendre et appliquer la technique VLSM	4
5	Lire et comprendre l'implémentation Python de ces calculs	5
6	S'exercer sur des cas concrets corrigés	6 et 7

■ Prérequis : aucune connaissance réseau préalable n'est requise.

Une connaissance de base du binaire est un plus

Pour la partie Python , une connaissance élémentaire de Python est utile.

2. Rappels : Le Système Binaire

Les réseaux IPv4 fonctionnent en **binaire** (base 2). Un octet est un groupe de 8 bits, chacun valant 0 ou 1. Comprendre le binaire est essentiel pour saisir le fonctionnement du masque de sous-réseau.

2.1 Conversion décimal ↔ binaire

Chaque bit d'un octet a une valeur en puissance de 2 :

Bit n°	7	6	5	4	3	2	1	0
Valeur	128	64	32	16	8	4	2	1
Exemple (192)	1	1	0	0	0	0	0	0

192 en binaire = 128+64 = 11000000

2.2 Les opérations bit à bit — ET et OU logique

Deux opérations sont au cœur des calculs réseau :

ET logique (AND, &) : le résultat est 1 **seulement si les deux bits sont 1**.

OU logique (OR, |) : le résultat est 1 **si au moins un des deux bits est 1**.

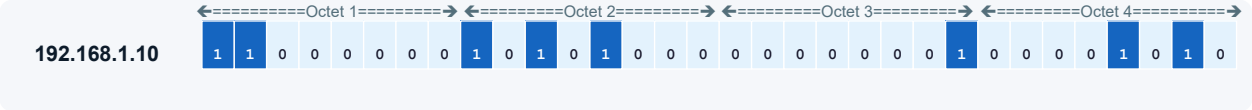
Bit A	Bit B	A AND B (ET)	A OR B (OU)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Tableau récapitulatif des opérations bit à bit

3. Concepts Fondamentaux du Réseau IPv4

3.1 L'adresse IP

Une adresse **IPv4** est composée de **32 bits**, organisés en **4 groupes de 8 bits (octets)**, séparés par des points. Chaque octet représente une valeur décimale entre **0 et 255**.



Représentation binaire de l'adresse 192.168.1.10

3.2 Le masque de sous-réseau

Le **masque de sous-réseau** est également une valeur de 32 bits. Il indique quelle partie de l'adresse IP désigne le **réseau** (bits à 1) et quelle partie désigne l'**hôte** (bits à 0).

Propriété fondamentale : dans un masque, tous les bits à 1 sont *toujours* placés en tête, suivis de tous les bits à 0. Il n'y a jamais de mélange.

Masque /24 : 24 bits à 1 (réseau) suivis de 8 bits à 0 (hôtes)

3.3 La notation CIDR

La notation **CIDR** (*Classless Inter-Domain Routing*) exprime le masque par le **nombre de bits à 1**, précédé du signe */*.

CIDR = 32 – nombre de bits de la partie hôte

Comment trouver le nombre de bits de la partie hôte ?

Soit n le nombre de bit de la partie hôte (n est un entier positif).

Pour trouver **n**, on cherche sa plus petite valeur tel que 2^n soit supérieur ou égal au nombre d'adresses nécessaires (hôtes + 2 adresses réservées : l'adresse réseau et de diffusion).

Nbre d'adresses = nbre d'hôtes + 2

2^n >= nbre d'adresses ➔ on incrémente n jusqu'à satisfaire cette condition

■ Exemple : pour 50 hôtes, on a besoin de 50+2 = 52 adresses.
2^6 = 64 ≥ 52 ➔ 6 bits hôte ➔ CIDR = 32 - 6 = /26
Masque = 255 . 255 . 255 . 192 (les 26 premiers bits sont à 1)

Tableau récapitulatif du masque et du CIDR

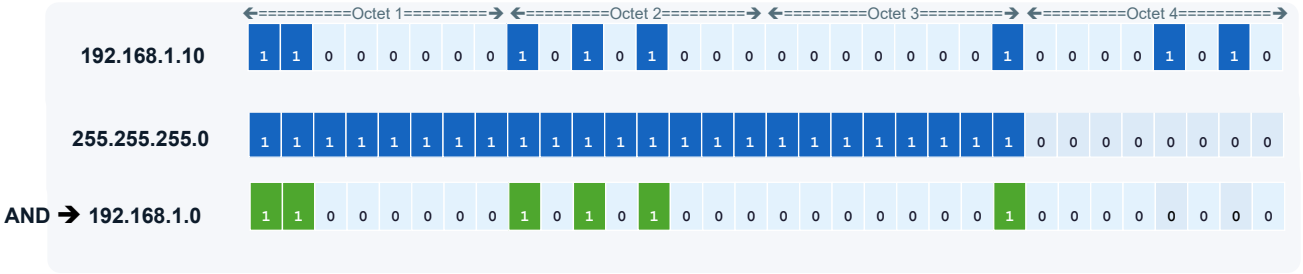
CIDR	Masque décimal	Représentation binaire	Hôtes max
/24	255.255.255.0	11111111.11111111.11111111.00000000	254
/25	255.255.255.128	11111111.11111111.11111111.10000000	126
/26	255.255.255.192	11111111.11111111.11111111.11000000	62
/27	255.255.255.224	11111111.11111111.11111111.11100000	30
/28	255.255.255.240	11111111.11111111.11111111.11110000	14
/29	255.255.255.248	11111111.11111111.11111111.11111000	6
/30	255.255.255.252	11111111.11111111.11111111.11111100	2

3.4 L'adresse réseau — Opération ET logique (AND)

L'**adresse réseau** est la première adresse du réseau. Elle identifie le réseau lui-même et ne peut pas être assignée à un hôte. On l'obtient en appliquant un **ET logique bit à bit** entre une **adresse IP** et le **masque**.

Adresse réseau = (Adresse IP) AND Masque

Exemple : Déterminer l'adresse réseau de la machine qui a l'adresse **192.168.1.10** avec le masque **255.255.255.0 (/24)** :



L'adresse réseau (ET logique)

■ Remarque : les octets ou le masque est 255 (11111111) ne changent pas, car X AND 255 = X.

Ainsi la machine avec l'adresse **192.168.1.10 /24** appartient au réseau **192.168.1.0**

3.5 L'adresse de diffusion (Broadcast) — Opération OU logique

L'**adresse de diffusion** est la dernière adresse du réseau. Un paquet envoyé à cette adresse est reçu par **tous les hôtes** du réseau. On l'obtient avec un **OU logique** entre l'**IP** et le **complément du masque**.

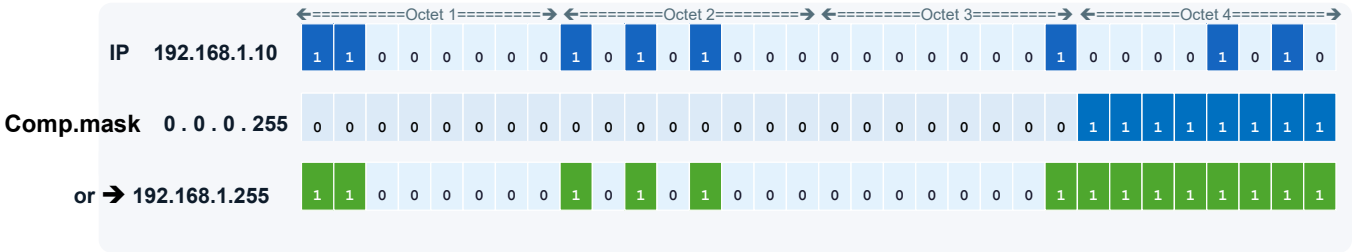
Adresse diffusion = (Adresse IP) OR (complément du masque)

Complément du masque = 255 - masque

Exemple : Déterminer l'adresse de diffusion du réseau précédemment calculer.

Rappel : la machine avait comme adresse **192.168.1.10** avec le masque **255.255.255.0 (/24)** :

Le complément du masque se calcule de la manière suivante : pour chaque octet du masque, on effectue l'opération **255 – la valeur de l'octet**. Ainsi le complément de 255.255.255.0 est **0.0.0.255**



L'adresse de diffusion (OU logique)

> ■ Remarque : Calculer l'adresse de diffusion revient tout simplement à mettre les bits de la partie hôte de l'adresse du réseau à 1

Ainsi l'adresse de diffusion est : **192.168.1.255**

3.6 Plage d'adresses utilisables

Les adresses utilisables par les équipements (PC, routeurs, etc.) sont toutes les adresses entre l'adresse réseau et l'adresse de diffusion, ces deux extrêmes étant **réservées**.

Adresse	Valeur	Rôle	Assignable ?
Adresse réseau	192.168.1.0	Identifie le réseau	Non
Première adresse utilisable	192.168.1.1	1er hôte possible	Oui
...	...	Hôtes intermédiaires	Oui
Dernière adresse utilisable	192.168.1.254	Dernier hôte possible	Oui
Adresse de diffusion	192.168.1.255	Broadcast réseau	Non

Nombre d'adresses utilisables = 2^n - 2 (n = nbre bits de la partie hôte)

4. Le VLSM — Découpage en Sous-réseaux

Le **VLSM** (*Variable Length Subnet Masking*) est une technique qui permet de diviser un réseau IP en plusieurs **sous-réseaux de tailles différentes**, selon les besoins réels de chaque segment du réseau.

4.1 Pourquoi le VLSM ?

Sans VLSM, on devait attribuer le même masque à tous les sous-réseaux, ce qui gaspille souvent des adresses. Avec le VLSM, chaque sous-réseau reçoit exactement le nombre d'adresses dont il a besoin.

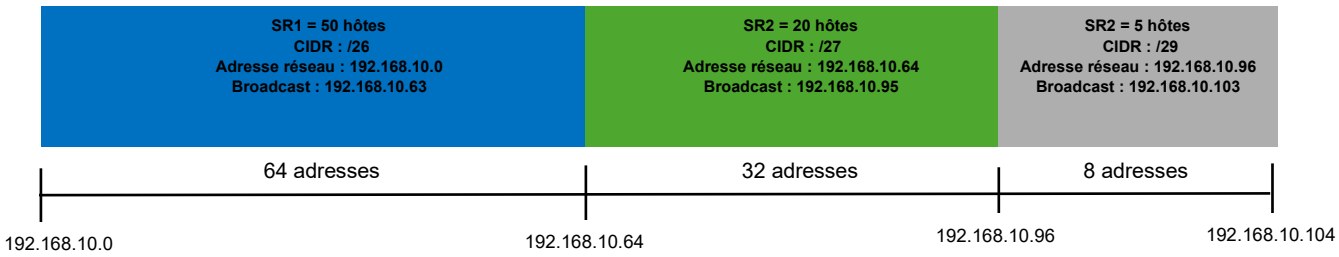
	Sans VLSM	Avec VLSM
Méthode	Même masque partout	Masque adapté à chaque besoin
Gaspillage	Important	Minimal
Complexité	Simple	Plus technique
Utilisation	Petits réseaux simples	Réseaux professionnels

4.2 Méthode de découpage VLSM — Étape par étape

- **Étape 1 :** Lister tous les sous-réseaux et leurs besoins en hôtes
- **Étape 2 :** Trier les besoins des sous-réseaux en hôtes (du plus grand au plus petit)
- **Étape 3 :** Pour chaque sous-réseau, calculer le CIDR nécessaire
- **Étape 4 :** Attribuer l'espace d'adressage séquentiellement
- **Étape 5 :** Le prochain sous-réseau commence à l'adresse qui suit la diffusion du réseau précèdent

4.3 Exemple de découpage — Réseau 192.168.10.0

Soit 3 sous-réseaux à créer à partir de **192.168.10.0** avec les besoins suivants :
- SR1 = 50 hôtes ;
- SR2 = 20 hôtes ;
- SR3 = 5 hôtes.



Découpage VLSM de 192.168.10.0 — allocation séquentielle du plus grand au plus petit

SR	Hôtes requis	Bits hôte	CIDR	Masque	Adresse réseau	Diffusion	Plage utilisable
SR1	50	6 (2^6=64≥52)	/26	255.255.255.192	192.168.10.0	192.168.10.63	.1 → .62
SR2	20	5 (2^5=32≥22)	/27	255.255.255.224	192.168.10.64	192.168.10.95	.65 → .94
SR3	5	3 (2^3=8≥7)	/29	255.255.255.248	192.168.10.96	192.168.10.103	.97 → .102

```
192.168.10.0
├── SR1  192.168.10.0/26
├── SR2  192.168.10.64/27
└── SR3  192.168.10.96/29
```

> ■ Important : chaque sous-réseau commence à l'adresse de diffusion du sous-réseau précédent + 1.
> SR2 commence en .64 car SR1 se termine en .63 (diffusion).

5. Implémentation Python — Netcalc

Cette section explique comment les calculs théoriques vus précédemment ont été traduits en Python. Le programme est structuré en **fonctions indépendantes**, chacune ayant une responsabilité unique.

5.1 Architecture du programme

Fonction	Rôle	Section
`ip_valid()`	Valide le format d'une adresse IP	5.2
`check_entier_positif()`	Vérifie qu'une saisie est un entier > 0	5.2
`saisir_nbre_sous_reseau()`	Demande le nombre de sous-réseaux (VLSM)	5.2
`saisir_hote_chaque_sous_reseau()`	Collecte les besoins en hôtes (VLSM)	5.2
`mask_en_decimal()`	Calcule le masque à partir du CIDR	5.3
`conv_mask_decimal_vers_binaire()`	Convertit le masque en binaire	5.3
`calcul_info_sous_reseau()`	Calcule toutes les infos d'un sous-réseau	5.4
`reseau_simple()`	Effectue le traitement pour un réseau simple	5.5
`check_espace_adressage_suffisant()`	Vérifie que le CIDR permet de couvrir les besoins de tous les sous-réseaux (VLSM)	5.6
`calcul_sous_reseau_suivant()`	Calcule l'adresse du prochain sous-réseau(VLSM)	5.6
`vlsm()`	Orchestre le traitement pour le VLSM	5.6
`formater_liste()`	Formate une liste d'entiers en chaîne	5.7
`affichage_infos_sous_reseau()`	Affiche tous les résultats	5.7
`main()`	Menu principal et orchestration	5.8

5.2 Fonctions de saisie et de validation

Avant tout calcul, le programme s'assure que les données saisies sont correctes. Ces fonctions forment le **bouclier d'entrée** du programme.

Python

```python
def ip_valid(adr):   #Vérification : exactement 4 octets, numériques, compris entre 0 et 255
    return len(adr) == 4 and all(octet.isdigit() for octet in adr) and all(0<=int(octet)<=255 for octet in adr)

def check_entier_positif(nbre):
    return nbre.isdigit() and int(nbre) > 0     #isdigit() => que des entiers positifs(ex: rejette '-5' ou '3.5')

def saisir_nbre_sous_reseau():
    while True:     #Saisie et validation du nombre de sous-réseau
        nbre_sous_reseau = input("Saisissez le nombre de sous-reseau: ")
        if check_entier_positif(nbre_sous_reseau):
            break
        print("Le nombre de sous reseau doit être un entier supérieur à 0!")
    return int(nbre_sous_reseau)

def saisir_hote_chaque_sous_reseau(nbre_sous_reseau):      #Saisie le nombre d'hotes de chaque sous-réseau
    nbre_hote_sr = []
    for i in range(nbre_sous_reseau):
        while True:
            saisie = input(f"Saisissez le nombre d'hôte du sous-reseau {i+1}: ")
            if check_entier_positif(saisie):
                nbre_hote_sr.append(int(saisie))
                break
            print("Le nombre d'hôte doit être un entier supérieur à 0! ")
    return nbre_hote_sr     #Liste d'entiers(ex : [12,3,43,200])
```

Fonctions de saisie et de validation

- all() retourne True seulement si la condition est vraie pour **Tous** les éléments.
  .split('.') découpe '192.168.1.0' en ['192','168','1','0'] (liste de chaine).
  isdigit() retourne False pour les nombre négatifs et décimaux

5.3 Calcul du masque — mask_en_decimal() et conv_mask_decimal_vers_binaire()

Ces deux fonctions calculent le masque sous ses deux formes : décimale (255.255.255.192) et binaire (11111111.11111111.11111111.11000000).

Python

```python
def mask_en_decimal(cidr):   # Calcul du masque sous format decimal
    mask_decimal = []
    for octet in range(4):      #On remplit les 4 octets 1 à 1
        if (cidr >= 8 ):    #L'octet est entièrement dans le réseau(donc il vaut 255)
            mask_decimal.append(255)
            cidr -= 8
        elif(cidr > 0):     #L'octet est partiellement dans le réseau(déterminer la partie qui n'est pas dans le réseau)
            bits_hote = 8 - cidr       #Nbre de bits hote dans cet octet
            mask_decimal.append(256 - (2**bits_hote))
            cidr = 0
        else:       #L'octet est entiérement hote(donc il vaut 0)
            mask_decimal.append(0)
    return mask_decimal

def conv_mask_decimal_vers_binaire(adr): # Convertir une adresse de decimal vers binaire
    return [format(octet,'08b') for octet in adr]   #Convertis chaque octet en binaire et l'affiche sur 8 bits
```

Fonction calcul du masque décimal et conversion binaire

- Exemple d'exécution pour CIDR = /26 :

  Octet 1 : cidr = 26 ≥ 8 ➔ mask[0] = 255, cidr = 18

  Octet 2 : cidr = 18 ≥ 8 ➔ mask[1] = 255, cidr = 10

  Octet 3 : cidr = 10 ≥ 8 ➔ mask[2] = 255, cidr = 2

  Octet 4 : cidr = 2 < 8 ➔ mask[3] = 256 - 2^6 = 192, cidr = 0

  Résultat : [255, 255, 255, 192] ➔ 255.255.255.192

- **format(octet, '08b') for octet in adr** permet de convertir chaque octet en binaire et sous format de 8 bit.

  Par exemple la conversion de 5 en binaire, donnerait '00000101' au lieu de '101'

5.4 Fonction centrale — calcul_info_sous_reseau()

C'est le **cœur du programme**. Elle reçoit une adresse IP et un nombre d'hôtes, effectue tous les calculs et retourne un dictionnaire complet.

Python

```python
def calcul_info_sous_reseau(add_ip, nbre_hote_voulu):
    #Etape 1 : calculer le nombre de bits hôte nécessaires
    nbre_adresse = int(nbre_hote_voulu) + 2      # +2 pour l'adresse réseau et l'adresse de diffusion
    nbre_bits_hostID =0
    while((2 ** nbre_bits_hostID) < nbre_adresse):
        nbre_bits_hostID += 1

    #Etape 2 : déduire le CIDR et le nombre d'hôtes réel
    CIDR = 32 - nbre_bits_hostID
    nbre_adr_utlisable = (2**nbre_bits_hostID) -2

    #Etape 3 : calculer le masque
    mask_decimal = mask_en_decimal(CIDR)
    mask_binaire = conv_mask_decimal_vers_binaire(mask_decimal)

    #Etape 4 : calculer l'adresse réseau (ET logique)
    reseau = []
    for i in range(4):
        reseau.append(add_ip[i] & mask_decimal[i])

    #Etape 5 : calculer l'adresse de diffusion (OU logique)
    diffusion = []
    for i in range(4):
        diffusion.append(add_ip[i] | (255 - mask_decimal[i]))

    #Etape 6 : calculer la première adresse adresse utilisable
    premiere = reseau.copy()
    premiere[3] += 1
    #Etape 7 : calculer la dernière adresse utilisable
    derniere = diffusion.copy()
    for i in range(3, -1, -1):      #Parcourir de droite à gauche
        if derniere[i] > 0:         #et décrémenter le premier octet != 0
            derniere[i] -= 1
            break
    #Etape 8 : retourner un dictionnaire avec tous les paramétres réseaux
    return {
        "nbre_hote"             : nbre_hote_voulu,      "CIDR"                  : CIDR,
        "mask_decimal"          : mask_decimal,         "mask_binaire"          : mask_binaire,
        "reseau"                : reseau,               "diffusion"             : diffusion,
        "premiere"              : premiere,             "derniere"              : derniere,
        "nbre_adr_utilisable"   : nbre_adr_utlisable,
    }
```

Fonction calcul_info_sous_reseau() --- cœur du programme

- range(3, -1, -1) parcourt les indices 3, 2, 1, 0 (de droite à gauche).
  Cela permet de décrémenter correctement même quand le dernier octet est 0.
  Ex : diffusion = 192.168.2.0 ➔ on décrémente l'octet à l'indice 2 : 192.168.1.255

5.5 Un sous-reseau — reseau_simple()

Cette fonction est appelée lorsque l'utilisateur souhaite faire des calculs pour un seul sous-réseau (réseau simple). Elle demande une adresse IP et un nombre d'hôtes pour ensuite effectuer les calculs réseaux et afficher les résultats

.

Python

```python
def reseau_simple():
    while True:     # Verifier que le format de l'adresse est valide(avec les fonction adr_est_entier, long_valid et octet_entre_0_255)
        add_ip = []
        add_ip = input("\nVeuillez saisir votre adresse réseau ou l'adresse d'un hôte de votre réseau: ").strip().split(".")
        if ip_valid(add_ip):
            break
        print("Format de l'adresse invalide, (exemple d'adresse valide: 192.168.1.4). Veuillez reessayer!")
    add_ip = [int(octet) for octet in add_ip]                          # Convertir chaque octet de l'adresse ip en entier(de base ce sont des chaines de caracteres)
    while True:
        nbre_hote = input("Combien d'hôtes voulez vous dans le reseau? :")
        if check_entier_positif(nbre_hote):
            break
        print("Le nombre d'hôtes doit être un entier supérieur à 0!")
    info = calcul_info_sous_reseau(add_ip, nbre_hote)
    print("\n==================== Infos de votre réseau =======================")
    affichage_infos_sous_reseau(info)
```

Fonction reseau_simple() --- un seul sous-réseau

- L'adresse IP est d'abord recupere sous la forme d'une liste de chaine de caracteres avant d'etre convertie en une liste d'entier par
  add_ip = [int(octet) for octet in add_ip]

5.6 Plusieurs sous-réseaux (VLSM) — check_espace_adressage_suffisant(), calcul_sous_reseau_suivant() et vlsm()

Ces 3 fonctions effectuent le traitement VLSM. Après chaque allocation, la fonction **calcul_sous_reseau_suivant**() trouve la **prochaine adresse réseau disponible** en ajoutant 1 à l'adresse de diffusion du sous-réseau précèdent.

Python

```python
def check_espace_adressage_suffisant(cidr, nbre_hote_sr):
    capacite_cidr = 2 ** (32 - cidr)
    som_bloc_adresse = 0
    for hote in nbre_hote_sr:
        i = 0
        while(2 ** i) < (hote + 2):
            i += 1
        som_bloc_adresse += (2 ** i)
    return capacite_cidr >= som_bloc_adresse

def calcul_sous_reseau_suivant(diffusion_prec):
    reseau = diffusion_prec.copy()
    for i in range(3, -1, -1):      #On part de l'octet le plus à droite
        if reseau[i] < 255:
            reseau[i] += 1      #On peut incrémenter
            break
        else:
            reseau[i] = 0   #Incrémentation impossible: on met l'octet à 0 et on passe à l'octet suivant(de la droite vers la gauche)
    return reseau


def vlsm():
    while True:
        while True:
            try:
                adresse, cidr = input("\nSaisissez l'adresse à utiliser pour faire le découpage ainsi que le CIDR(Exemple de saisie valide: 12.14.10.0/24) : ").strip().split("/")
                adresse = adresse.split(".")
                if ip_valid(adresse) and check_entier_positif(cidr) and int(cidr)<32:
                    break
                print("Format de l'adresse ou du CIDR invalide, veuillez reessayez")
            except:
                print("Saisie invalide(Exemple de saisie valide: 12.14.10.0/24), veuillez reessayer")
        adresse = [int(octet) for octet in adresse]
        cidr = int(cidr)
        nbre_sous_reseau = saisir_nbre_sous_reseau()                        # Saisir le nombre de sous reseau pour le decoupage
        nbre_hote_sr = saisir_hote_chaque_sous_reseau(nbre_sous_reseau)     # Saisir le nbre d'hote de chaque sous reseau
        if check_espace_adressage_suffisant(cidr, nbre_hote_sr):    #Vérifier que le CIDR saisie par le user permet de couvrir tous les besoins
            break
        print("Le CIDR choisi ne permet pas de remplir les besoins de tous les sous-réseau, veuillez choisir un CIDR plus faible ou diminuer le nombre d'hotes des sous-réseaux")
    nbre_hote_sr = sorted(nbre_hote_sr,reverse=True)                    # Trier le nbre d'hote des sous != sous reseaux par ordre decroissant
    sous_reseau_actuel = adresse.copy()
    print("======================================= VLSM =======================================")
    for i, hote in enumerate(nbre_hote_sr):
        print(f"\n============================ Sous-reseau {i+1} ==============================")
        info = calcul_info_sous_reseau(sous_reseau_actuel,hote)
        affichage_infos_sous_reseau(info)
        sous_reseau_actuel = calcul_sous_reseau_suivant(info['diffusion'])
```

Fonctions VLSM --- plusieurs sous-réseaux

■ Exemple de découpage VLSM pour 192.168.10.0/24 en 3 sous-réseau(100 hôtes, 58 hôtes, 30 hôtes).

1) D'abord on vérifie avec la fonction **check_espace_adressage_suffisant()** que ce CIDR(/24) permet de couvrir les besoins de tous les sous-réseaux.

/24 permet d'avoir 256 adresses

SR 1 : 100 hôtes ➔ /25, bloc adresses = 128
SR 2 : 58 hôtes ➔ /26, bloc adresses = 64
SR 3 : 30 hôtes ➔ /27, bloc adresses = 32

la somme des blocs d'adresse = 224 ≤ 256 donc le /24 permet effectivement de couvrir tous les besoins

2) Ensuite on peut poursuivre avec l'enchainement des sous-réseaux

SR 1 : réseau = 192.168.10.0, diffusion = 192.168.10.63
calcul_sous_reseau_suivant(.63) ➔ .63 < 255 ➔ +1 ➔ 192.168.10.64
SR 2 : réseau = 192.168.10.64, diffusion = 192.168.10.95
calcul_sous_reseau_suivant(.95) ➔ .95 < 255 ➔ +1 ➔ 192.168.10.96
SR 3 : réseau = 192.168.10.96, diffusion = 192.168.10.103

Maintenant supposons nous avons SR n : diffusion = 192.168.10.255
calcul_sous_reseau_suivant(.255) ➔ .255 = 255 ➔ on ne peut incrémenter, le .255 devient .0 et on passe à l'octet qui le précède(.10).
calcul_sous_reseau_suivant(.10) ➔ .10 < 255 ➔ +1 ➔ 192.168.11.0
SR n+1 : réseau = 192.168.11.0, diffusion = 192.168.11….

5.7 Fonctions d'affichage — formater_liste() et affichage_infos_sous_reseau()

Ces fonctions transforment les listes d'entiers en chaînes lisibles et affichent tous les résultats de manière organisée.

Python

```python
def formater_liste(liste):
    return ".".join(map(str,liste))     #Formater les listes pour préparer l'affichage

def affichage_infos_sous_reseau(info):
        #Affichage des infos du sous-reseau
        print(f"Nombre d'hotes                 : {info['nbre_hote']}")
        print(f"CIDR                           : /{info['CIDR']}")
        print(f"Masque en decimal              : {formater_liste(info['mask_decimal'])}")
        print(f"Masque en binaire              : {".".join(info['mask_binaire'])}")
        print(f"Adresse reseau                 : {formater_liste(info['reseau'])}")
        print(f"Adresse de diffusion           : {formater_liste(info['diffusion'])}")
        print(f"Premiere adresse utilisable    : {formater_liste(info['premiere'])}")
        print(f"Derniere adresse utilisable    : {formater_liste(info['derniere'])}")
        print(f"Plage des adresses utilisables : {formater_liste(info['premiere'])} à {formater_liste(info['derniere'])}")
        print(f"Nombre d'adresse utilisable    : {info['nbre_adr_utilisable']}\n\n")
```

Fonction d'affichage

- map(str,liste) : convertit chaque entier de liste en chaine de caractères.
  ' . '.join() : assemble les éléments de la liste avec « . » comme séparateur

NB : .join() ne fonctionne qu'avec des chaines de caractères d'où la fonction map(str,liste)

5.8 Menu principal — main()

La fonction **main()** orchestre tout le programme via un menu en boucle.

Python

```python
def main():
    print("Bienvenu sur Netcalc.")
    while True:     #Boucle : l'utilisateur peut faire plusieurs calculs de suite
        print("========================= MENU =======================")
        print("1 - Un sous-reseau(reseau simple)")
        print("2 - VLSM(plusieurs sous-reseaux de tailles differentes)")
        print("3 - Quitter")
        mode = input("Choisir un mode(1, 2 ou 3) : ").strip()
        if mode == "1":     #Un seul sous réseau
            reseau_simple()
        elif mode == "2":   #Enchainement VLSM
            vlsm()
        elif mode == '3':   #Quitter
            break
        else:
            print("Choix invalide, choisir entre 1, 2 et 3")
    print("Nous vous remercions d'avoir utiliser Netcalc, à bientôt")
if __name__ == "__main__":
    main()
```

Fonction main() – menu et orchestration générale

6. Exemples d'utilisation de Netcalc

Exemple 1 — Réseau simple (mode 1)

Énoncé : Calculer les informations du réseau contenant l'adresse **10.0.5.37** et ayant 30 hôtes.

Paramètre	Calcul	Résultat
Hôtes requis	—	**30**
Bits hôte	2^5 = 32 ≥ 32 => n=5	**5 bits**
CIDR	32 - 5	**/27**
Masque	octet4 : 256 - 2^3 = 256 - 8 = 248... /27	**255.255.255.224**
Adresse réseau	10.0.5.37 AND 255.255.255.224	**10.0.5.32**
Adresse diffusion	10.0.5.37 OR 0.0.0.31	**10.0.5.63**
Première adresse utilisable	10.0.5.32 + 1	**10.0.5.33**
Dernière adresse utilisable	10.0.5.63 - 1	**10.0.5.62**
Hôtes disponibles	2^5 - 2	**30**

Python

```
PS D:\Cheikh\Etudes\Projets\vs_code\calculatrice_reseau> python .\calculatrice_reseau_by_cheikh.py
Bienvenu sur Netcalc.
========================= MENU =======================
1 - Un sous-reseau(reseau simple)
2 - VLSM(plusieurs sous-reseaux de tailles differentes)
3 - Quitter
Choisir un mode(1, 2 ou 3) : 1

Veuillez saisir votre adresse réseau ou l'adresse d'un hôte de votre réseau: 10.0.5.37
Combien d'hôtes voulez vous dans le reseau? :30

====================== Infos de votre réseau =======================
Nombre d'hotes                  : 30
CIDR                            : /27
Masque en decimal               : 255.255.255.224
Masque en binaire               : 11111111.11111111.11111111.11100000
Adresse reseau                  : 10.0.5.32
Adresse de diffusion            : 10.0.5.63
Premiere adresse utilisable     : 10.0.5.33
Derniere adresse utilisable     : 10.0.5.62
Plage des adresses utilisables  : 10.0.5.33 à 10.0.5.62
Nombre d'adresse utilisable     : 30
```

Démonstration pour un réseau simple

Exemple 2 — VLSM complet (mode 2)

Énoncé : Découper **172.16.0.0** pour 2 services : Direction (100 hôtes), Informatique (50 hôtes).

Ordre	Service	Hôtes	Bits	CIDR	Adresse réseau	Diffusion	Plage
1	Direction	100	7 (128≥102)	/25	172.16.0.0	172.16.0.127	.1→.126
2	Informatique	50	6 (64≥52)	/26	172.16.0.128	172.16.0.191	.129→.190

Python

```
PS D:\Cheikh\Etudes\Projets\vs_code\calculatrice_reseau> python .\calculatrice_reseau_by_cheikh.py
Bienvenu sur Netcalc.
========================= MENU =======================
1 - Un sous-reseau(reseau simple)
2 - VLSM(plusieurs sous-reseaux de tailles differentes)
3 - Quitter
Choisir un mode(1, 2 ou 3) : 2

Saisissez l'adresse à utiliser pour faire le découpage ainsi que le CIDR(Exemple de saisie valide: 12.14.10.0/24) : 172.16.0.0/24
Saisissez le nombre de sous-reseau: 2
Saisissez le nombre d'hôte du sous-reseau 1: 100
Saisissez le nombre d'hôte du sous-reseau 2: 50
==================================== VLSM ====================================

============================ Sous-reseau 1 ==============================
Nombre d'hotes                  : 100
CIDR                            : /25
Masque en decimal               : 255.255.255.128
Masque en binaire               : 11111111.11111111.11111111.10000000
Adresse reseau                  : 172.16.0.0
Adresse de diffusion            : 172.16.0.127
Premiere adresse utilisable     : 172.16.0.1
Derniere adresse utilisable     : 172.16.0.126
Plage des adresses utilisables  : 172.16.0.1 à 172.16.0.126
Nombre d'adresse utilisable     : 126



============================ Sous-reseau 2 ==============================
Nombre d'hotes                  : 50
CIDR                            : /26
Masque en decimal               : 255.255.255.192
Masque en binaire               : 11111111.11111111.11111111.11000000
Adresse reseau                  : 172.16.0.128
Adresse de diffusion            : 172.16.0.191
Premiere adresse utilisable     : 172.16.0.129
Derniere adresse utilisable     : 172.16.0.190
Plage des adresses utilisables  : 172.16.0.129 à 172.16.0.190
Nombre d'adresse utilisable     : 62
```

Démonstration pour plusieurs sous-réseaux(VLSM)

7. Exercices Corrigés

Ces exercices permettent de vérifier la compréhension des concepts.

Exercice 1 — Calcul simple

■ Données : IP = 192.168.5.200, nombre d'hôtes requis = 12.
Question : Calculez le CIDR, le masque, l'adresse réseau, l'adresse de diffusion et la plage utilisable.

Correction :

Étape 1 — Bits hôte	12 hôtes + 2 = 14 adresses => 2^4=16 ≥ 14 => n=4 bits
Étape 2 — CIDR	32 - 4 = /28
Étape 3 — Masque	28 bits de 1 suivis de 4 bits de 0 => 255.255.255.240
Étape 4 — Réseau	192.168.5.200 AND 255.255.255.240 Adresse réseau = 192.168.5.192
Étape 5 — Diffusion	192.168.5.200 OR 0.0.0.15 Adresse diffusion = 192.168.5.207
Plage utilisable :	192.168.5.193 à 192.168.5.206 (14 hôtes)

Exercice 2 — Interprétation du masque binaire

■ Données : Masque binaire = 11111111.11111111.11111000.00000000
Question 1 : Convertissez ce masque en décimal.
Question 2 : Quel est le CIDR correspondant ?
Question 3 : Combien d'hôtes ce réseau peut-il accueillir ?

Correction :

- ➤ Q1 : Octet 3 : 11111000 = 128+64+32+16+8 = 248 => Masque = 255.255.248.0
- ➤ Q2 : On compte les '1' : 8+8+5 = 21 bits à 1 => CIDR = /21
- ➤ Q3 : Bits hôte = 32 - 21 = 11 => 2^{11} - 2 = 2048 - 2 = 2046 hôtes

Exercice 3 — VLSM

- Adresse de base : 10.10.0.0

  Besoins : Réseau A = 60 hôtes, Réseau B = 25 hôtes, Réseau C = 10 hôtes.

  Question : Effectuez le découpage VLSM complet.

Correction :

Réseau	Hôtes	Calcul bits	CIDR	Masque	Adresse réseau	Diffusion
A	60	2^6=64≥62 => n=6	/26	255.255.255.192	10.10.0.0	10.10.0.63
B	25	2^5=32≥27 => n=5	/27	255.255.255.224	10.10.0.64	10.10.0.95
C	10	2^4=16≥12 => n=4	/28	255.255.255.240	10.10.0.96	10.10.0.111

8. Conclusion

Ce document a présenté les principes fondamentaux du sous-réseautage IPv4 ainsi que leur implémentation en python.
L'objectif était de montrer comment des concepts théoriques peuvent être traduits en algorithmes et en programme informatique.